Postgres managed by ClickHouse | One stack for transactions and analytics →

Back

Resources / Engineering

The ultimate guide to Open Source Observability in 2026: From silos to stacks

Manveer Chawla
Last updated: Nov 10, 2025

You've been told the three pillars of observability, logs, metrics, and traces, are the answer. But stitching together separate, best-of-breed tools has likely left you with data silos, slow queries, and a constant battle against rising infrastructure costs. During an incident, you're not debugging. Instead, you're manually correlating timestamps across three different UIs. This isn't a sustainable strategy.

The most effective and cost-efficient observability solution for 2026 isn't a collection of disparate tools. It's a unified, open-source **stack** built on a powerful data engine. This guide provides the architectural blueprints to help you understand why this shift is happening and how to build your stack the right way.

TL;DR

- The "three pillars" (logs, metrics, traces) are just data types, not a solution. This model led to separate, siloed tools (like Elasticsearch for logs, Prometheus for metrics) that are difficult to correlate and expensive.
- We compare the evolution of obs[illegible]ctures: from “search-fortress” and "best-of-breed" silos to the modern, cost-efficient "unified database" approach.

↑ Scroll to top

Postgres managed by ClickHouse | One stack for transactions and analytics →

- **ClickStack** is an opinionated, open-source, unified observability stack (OTel Collector, ClickHouse, HyperDX UI) engineered to solve the core problems of correlation, cost, and scale.

Why does the "three pillars" model lead to three silos?

The concept of "three pillars," logs, metrics, and traces, became popular in the mid-2010s as a way to categorize the essential data types for understanding a system's state. This model became popular as powerful, specialized open-source tools for each data type matured: **Prometheus** for metrics, the **ELK stack** for logs, and **Jaeger** for traces. This naturally led organizations to adopt a separate "best-of-breed" tool for each pillar, creating distinct data silos by default.

However, this approach has a fundamental flaw. The pillars represent raw data inputs, not a complete observability solution. The model leaves the complex and critical task of data analysis and correlation to you, the end-user. This is a task made nearly impossible by the siloed architecture it encourages.

This fragmentation creates tangible pain. During a critical incident, an engineer's workflow becomes a slow, manual, and error-prone process of "swivel-chair analysis." An SRE gets a metric-based alert in Grafana, pivots to Kibana to hunt for related error logs, and then pivots again to Jaeger, hoping to find a trace ID that connects the dots. This constant context-switching between different UIs and query languages increases Mean Time to Resolution (MTTR) and raises the risk of missing crucial connections between signals.While commercial observability platforms abstract this UI fragmentation behind a single interface, they typically introduce new challenges, namely expensive consumption-based pricing and vendor lock-in.

How does OpenTelemetry standardize data collection?

As of 2026, **OpenTelemetry (OTel)** has matured into the undisputed, vendor-neutral industry standard for instrumenting and transporting telemetry data. As the second most active project in the Cloud Native Computing Foundation (CNCF), its massive adoption is based on a core principle: the clear separation of data ingestion from backend storage and analysis.

Postgres managed by ClickHouse | One stack for transactions and analytics →

This modular design is a strategic advantage. It standardizes instrumentation, preventing vendor lock-in and giving you more flexibility. You can instrument your applications once with OTel and then route telemetry to any compatible backend simply by changing a configuration file. OTel perfectly solves the *ingestion* problem. With instrumentation standardized, the new bottleneck is the backend's ability to handle this massive flow of OpenTelemetry data. This leaves the most important question unanswered: **Where should you send your data, and how can you query it at scale without breaking the bank?** The answer lies in the architecture of your backend.

What are the three architectural blueprints for an Open-Source Observability stack?

The open-source observability landscape isn't a random collection of tools. It's an evolutionary journey. Each architectural pattern emerged to solve the problems of the last. Here are the three dominant blueprints, each with a litmus test to see where it breaks at scale.

Blueprint #1: The "search-fortress" (ELK/OpenSearch)

This blueprint is built on the ELK (Elasticsearch, Logstash, Kibana) or OpenSearch stack, which uses the Apache Lucene search library at its core.

Strengths: It is a fortress for unstructured, "Google-like" text search. Its **inverted index** makes it very effective for Security Information and Event Management (SIEM) and compliance use cases where analysts need to find a needle in a haystack of text.

The Breaking Point: For modern observability analytics, this architecture hits a wall.

- **Extremely high TCO:** The Lucene inverted index is notoriously inefficient, creating massive storage overhead. It's common for the index to be multiple times the size of the original data. Combined with poor compression, this leads to budget-breaking infrastructure costs. A 100TB/day workload can cost **$100,000+ per month** on Elasticsearch.

↑

Postgres managed by ClickHouse | One stack for transactions and analytics →

by `customer_id` for the last 7 days." These are not text searches, they are analytical aggregations. The Lucene-based stack performs poorly on these queries, especially over wide time ranges or on high-cardinality fields (e.g. `user_id` or `container_id`). These analytical queries cause high JVM memory pressure, leading to slow performance, query timeouts, and even `OutOfMemory` errors that crash nodes.

- **High operational complexity:** Managing an ELK cluster at scale is complex, often requiring a dedicated team of experts to handle shard management, capacity planning, and JVM tuning.

Blueprint #2: The "best-of-breed" siloed stack (LGTM)

This blueprint, often called the LGTM stack, uses specialized open-source tools for each signal: **Loki** for logs, **Grafana** for visualization, **Tempo** for traces, and **Mimir/Prometheus** for metrics.

Strengths: It uses top-tier projects, each highly optimized for its specific data type. Loki, in particular, dramatically lowers the cost of log storage compared to ELK by only indexing metadata labels, not the full log content.

The breaking point: While an improvement, this model introduces its own set of critical challenges.

- **High operational overhead:** You are now operating **three or more separate, stateful database systems**. This carries a significant hidden operational tax, requiring expertise in multiple distinct technologies and increasing engineering toil.
- **Cardinality and analytical gaps persist:** The problem just moves, it isn't solved. Prometheus is known to suffer from "cardinality explosion," which forces teams to rely heavily on pre-aggregation. This approach discards the raw data fidelity required for root-cause analysis, as you must anticipate your failure modes in advance. The problem then shifts to logs. Loki's cost-efficiency comes from its design of only indexing metadata labels, which makes its query performance on non-indexed log *content* slow by design. This restricts engineers to a very specific workflow (like finding logs for a known `trace_id`) and prevents the broad, exploratory analysis that is critical for finding unknown-unknowns. .

↑

Postgres managed by ClickHouse | One stack for transactions and analytics →

databases, there is no way to perform deep, analytical queries across the signals. An engineer cannot, for instance, write a single query to join metric spikes with specific log attributes and trace durations to find a common cause. This forces engineers into a rigid, pre-defined debugging path, making it difficult to investigate complex issues that do not fit that specific pattern.

Blueprint #3: The "unified database" stack (ClickHouse)

This modern architecture consolidates all telemetry, including logs, metrics, and traces, into a single, high-performance analytical database like ClickHouse.

Strengths: This is the most scalable, cost-effective, and flexible model. A single store for all telemetry eliminates data duplication, solves the high-cardinality problem at its root, and enables powerful, native correlation.

Why It's a Superior Solution:

- **Extremely low TCO:** ClickHouse's columnar storage and advanced compression achieve remarkable efficiency, using **10 times less storage space** than Elasticsearch. It also integrates natively with low-cost object storage (like Amazon S3) for long-term retention, drastically reducing costs.
- **Passes high-cardinality analytics:** The columnar architecture is purpose-built for this problem. Aggregating and filtering on high-cardinality data is a simple, sub-second `GROUP BY` query, not a cluster-threatening event. This is a primary driver for users optimizing high-volume logs or traces.
- **Passes on cross-signal correlation:** With all data in one place, correlation is native to the data engine itself, not a feature stitched together at the application or UI layer. This allows for highly efficient, deep analysis. You can join logs, metrics, and traces with standard SQL in a single query, allowing you to go from alert to root cause in seconds.
  - **A critical nuance**: While ClickHouse is exceptionally strong for logs and traces, it's important to be transparent about the current state of metrics support. It is excellent for general-purpose metric storage, but users deeply tied to the full PromQL ecosystem should be aware of current limitations in native PromQL compatibility. However, this area is evolving fast, with new enhancements being

Postgres managed by ClickHouse | One stack for transactions and analytics →

observability stack?

The success or failure of your observability stack depends on its data engine. The core challenge of modern observability is handling **high-cardinality data**, which explodes multiplicatively with every new service, server, or dimension you add (`application × server × code_path × user_id`). Search indexes and traditional time-series databases were not designed for this reality.

Here's how the underlying database technologies compare for the demands of observability analytics.

Feature	Elasticsearch (Search Index)	ClickHouse (Columnar Database)
Core architecture	**Inverted index** optimized for full-text search. Stores data in row-oriented JSON documents.	**Columnar storage** optimized for analytical aggregations. Stores data for each column together. Also supports inverted indices and bloom filters at a columnar level to accelerate textual searches.
Data compression	Poor. High storage overhead from the index and `doc_values` leads to significant data amplification.	Excellent. Requires at minimum 10 times less storage through superior compression codecs and columnar format.
High-cardinality aggregations	Slow and memory-intensive. Prone to `OutOfMemory` errors and query timeouts.	Extremely fast. Purpose-built for sub-second `GROUP BY` queries on trillions of rows.
Primary query language	KQL / Lucene. Powerful for text search, but less suited for complex analytical joins.	Standard SQL. A universal, powerful language for deep, cross-signal analysis and joins. Also supports Lucene Natural Language Search (transpiled to SQL) to ease migration from Elastic and Opensearch and provide a natural exploration language for logs.
Cost-efficiency (TCO)	Very High. Driven by massive storage, compute,	Very Low. Driven by extreme compression, efficient queries, and architectural simplicity.

Postgres managed by ClickHouse | One stack for transactions and analytics →

Elasticsearch is an excellent tool for searching text. But observability analytics, like calculating p99 latencies, grouping errors by customer ID, and finding outliers, are aggregation-heavy workloads. ClickHouse was built from the ground up for this exact task, making it a better architectural choice.

What is ClickStack, the pre-built unified stack?

ClickStack is the pre-built, open-source implementation of the "Unified Database" architecture. It provides an opinionated, end-to-end stack tuned for performance and cost-efficiency, consisting of a pre-configured **OpenTelemetry Collector** for ingestion, **ClickHouse** as the unified database, and **HyperDX** as the integrated UI.

This approach provides tangible, immediate benefits over a fragmented DIY approach.

Capability	DIY OSS Stack (Prometheus + ELK + Jaeger)	Unified OSS Stack (ClickStack)
Data correlation	**UI-Level & Rigid.** Correlation is limited to UI pivots (e.g., Grafana linking trace IDs to logs). Lacks native database-level joins across signals.	**Native & Deep.** All data is in one database. Correlation is done efficiently at the database layer, enabling deep, cross-signal analysis.
Data exploration	**Siloed & Slow.** Exploratory analysis is difficult. Traditional search stacks (ELK) are slow for analytics, and specialized log tools (Loki) are slow for searching non-indexed log content.	**Fast & Flexible.** Optimized for both broad trend analysis (fast `GROUP BY`s) and fast discovery (text search via inverted indices and bloom filters).
Cost at scale	High. Driven by the significant storage and compute footprint of multiple data stores.	Low. Up to 90% lower storage costs due to ClickHouse's high compression rates and efficient architecture.

↑

Postgres managed by ClickHouse | One stack for transactions and analytics →

performance	aggregations or high-cardinality metrics.	across massive datasets and fast text search (inverted indices, bloom filters)..
Maintenance overhead	Extremely High. Requires expertise to manage, scale, and secure at least three complex systems.	Dramatically Lower. A single, cohesive platform to manage, reducing operational complexity.

The power of this unified approach is proven by its use in some of the most demanding engineering organizations in the world.

> *"A lot of our peer companies are using ClickHouse for this exact use case. It's battle-tested and just the right tool for the job."*
>
> ***Tesla**, on building their quadrillion-row scale observability platform on ClickHouse.*

> *"Previously, querying the last 10 minutes would take 1–2 minutes. With ClickStack, it was just a case of how fast I could blink. The performance is real."*
>
> ***Character.ai**, after reducing log search times from minutes to milliseconds and cutting costs by 50% despite a 10x increase in log volume.*

> *"With ClickHouse, the database is green, queries are lightning-fast, and money is not on fire."*
>
> ***Anthropic**, on using ClickHouse to handle the "deluge of telemetry" from developing AI models like Claude 4.*

↑

Postgres managed by ClickHouse | One stack for transactions and analytics →

plugin. Many organizations successfully use Grafana on top of ClickHouse for metrics visualization, and it remains a powerful option for building dashboards to monitor known KPIs.

However, monitoring pre-defined dashboards is a different workflow from debugging an active incident. This distinction highlights the different design philosophies:

- **Grafana is for *dashboarding knowns*:** It excels at creating curated dashboards to monitor pre-defined metrics and Service Level Objectives (SLOs). Its strength lies in visualizing time-series data from one or many data sources.This design also encourages a rigid, metrics-first workflow (from an alert, to a trace, to logs) and is not built for the kind of exploratory, search-based analysis required to find unknown problems.
- **HyperDX is for *debugging unknowns*:** It is purpose-built for the investigative workflow required during an incident. The user experience is designed to move seamlessly between signals to find the root cause of novel problems, not just visualize known metrics.

While an engineer *can* use Grafana with ClickHouse for monitoring, the ClickStack observability platform includes HyperDX because it provides a cohesive, out-of-the-box debugging experience. In Grafana, ClickHouse is a data source *plugin*, not a native backend. This limits its integration into Grafana's core, opinionated workflows. Furthermore, any deep analysis in Grafana requires the engineer to write and optimize raw SQL, a task that is complex and unfamiliar to many SREs.

HyperDX, by contrast, is built for the unified database model. It offers native cross-signal correlation and abstracts this complexity, providing an intuitive Lucene-like syntax for search. An engineer can one-click from a specific log line to the exact distributed trace that generated it, or from a slow trace span to all the logs emitted during that operation. This is a native workflow, not a stitched-together experience.

Furthermore, HyperDX integrates other essential debugging tools, such as an intuitive Lucene-like syntax for log search, full Application Performance Monitoring (APM) trace waterfall views, and Real User Monitoring (RUM) features like session replay. These are core components of the UI, not just additional panels on a metrics dashboard. This

Postgres managed by ClickHouse | One stack for transactions and analytics →

No single tool is perfect for every job. Building trust means being honest about limitations. While the unified stack represents the future for large-scale observability, there are specific scenarios where older tools still excel.

- **Retain ELK/OpenSearch for relevance based search and SIEM:** For use cases where the primary requirement is not just finding text, but *ranking* it by relevance (like legal discovery or advanced SIEM threat hunting), Lucene's text-scoring engine remains the better choice. Modern observability platforms, including those using ClickHouse, also leverage inverted indices for fast, unstructured text search, but they are optimized for analytics and filtering, not relevance ranking.
- **Retain Prometheus for small to medium-scale metrics:** For environments where cardinality is well-controlled and the scale is manageable for a single server, Prometheus's simplicity, pull-based model, and powerful PromQL offer a straightforward and effective monitoring solution.
- **Use specialized tools for non-OTel-native use cases:** The ClickStack observability platform is focused on unifying the core signals (logs, metrics, traces) driven by OpenTelemetry at scale. For sub-use cases that fall outside this scope, such as universal profiling, deep database monitoring, or network monitoring, dedicated tools that provide the necessary out-of-the-box UI and collection agents are a better fit.

The strategic approach is not always to rip-and-replace, but to move new, high-volume, high-cardinality observability workloads to a unified stack while retaining specialized tools for the niche tasks they were designed for.

How do you get started: Stop juggling tools, start building a stack

The future of open-source observability isn't about which logging tool to choose; it's about building a unified stack on a database that can handle the scale and complexity of modern systems without compromising on cost or performance. The architectural shift from fragmented silos to a unified database is a direct response to the economic and technical limitations of previous generations of tools.

↑

Postgres managed by ClickHouse | One stack for transactions and analytics →

infrastructure.

- **Try the managed platform:** See the power without the setup. Try ClickStack on ClickHouse Cloud.
- **Join the community:** Have questions? Join a community of engineers building the future of observability.

Frequently asked questions

What are some open source observability stacks I can self-host?

When self-hosting, you generally have two architectural choices:

- **The "best-of-breed" siloed stack:** This is the popular **LGTM stack**, which stands for **L**oki (logs), **G**rafana (visualization), **T**empo (traces), and **M**imir/Prometheus (metrics). While each component is powerful, this approach carries a high operational tax. You become responsible for managing, scaling, and updating three or more separate, stateful database systems. Its most significant weakness is the lack of native cross-signal correlation, forcing your engineers back into "swivel-chair analysis" to debug incidents.
- **The "unified database" stack:** This is the modern, more efficient architecture. It consolidates all three signals into a single high-performance database. The leading open-source example is **ClickStack**, which combines the **OpenTelemetry Collector**, **ClickHouse** as the unified database, and **HyperDX** as the integrated UI. This model solves the correlation problem natively (you can join logs and traces with SQL) and dramatically lowers TCO and operational complexity by centralizing all telemetry data in one place.

What are some high-performance, scalable open-source alternatives to using Elasticsearch for an OpenTelemetry backend?

This is one of the most common challenges teams face at scale. Elasticsearch (and OpenSearch) is a "search-fortress" built on Lucene, which is excellent for full-text search but struggles with the demands of modern observability analytics which require the ability to perform aggregations to examine trends over time. Its inverted index leads to

Postgres managed by ClickHouse | One stack for transactions and analytics →

analytical database.

The leading open-source choice in this category is **ClickHouse**. It was purpose-built for the exact type of high-cardinality, high-volume analytical queries that observability requires. It provides:

- **Extreme compression:** Drastically reduces storage TCO.
- **Sub-second analytics:** Handles high-cardinality `GROUP BY` queries (e.g., "group errors by `user_id`") with ease.
- **SQL interface:** Uses a familiar, powerful query language, while also supporting Lucene for more exploratory log-based workflows

This is why ClickHouse is the data engine at the heart of ClickStack and is used by companies like Tesla, Character.ai, and Anthropic to power their observability platforms.

What are the most popular backends that offer native support for the full OpenTelemetry specification?

OpenTelemetry (OTel) is the industry standard for *collecting* and *transporting* data, but it doesn't store it. The OTel Collector can send data to many backends. The most popular open-source choices fall into two categories:

1. **Commercial SaaS platforms:** This includes major platforms like Datadog, Dynatrace, New Relic, and Splunk. They all support OTel ingestion to varying degrees, offering a managed, out-of-the-box experience. However, they are often the most expensive options, operate as "black boxes," and can lock you into their specific query languages and correlation UIs.
2. **Siloed backends (The LGTM Stack):** This involves using **Loki** for logs, **Mimir/Prometheus** for metrics, and **Tempo** for traces. While all are OTel-compatible, they are separate systems. This architecture perpetuates the "three silos" problem, making it difficult to analyze relationships *between* your signals.
3. **Unified backends (The ClickHouse Stack):** This architecture uses a single database, like **ClickHouse**, to store all three signals. ClickStack is the pre-built implementation of this. This is the only approach that natively supports full-stack correlation. You can

Postgres managed by ClickHouse | One stack for transactions and analytics →

simplified query experience while being compatible with open-source standards?

Most managed platforms are OTel-compatible, but the best ones are built on open-source foundations. This prevents vendor lock-in and ensures you're using a battle-tested engine.

The key is to look at the *architecture* the platform is built on. A modern managed platform should be built on a unified database to solve the core problems of scale, cost, and correlation.

This is the philosophy behind **ClickStack on ClickHouse Cloud**. It provides a fully managed platform that runs the open-source ClickStack (OTel, ClickHouse, HyperDX) for you. It directly delivers:

- **Scalable storage:** Uses ClickHouse's superior compression and ability to use low-cost object storage (like S3) for massive scale at a low cost.
- **Simplified query experience:** Provides a unified UI (HyperDX) for debugging and a powerful, standard **SQL** interface for deep analysis, eliminating the need to learn multiple, proprietary query languages.

Is a unified database good for all three signals, including logs, metrics, and traces?

Yes. The "wide event" model treats all telemetry as attributes of a single, context-rich event. A high-performance analytical database like ClickHouse is very good at storing and querying this wide, structured data. It can handle the high-volume, time-series nature of metrics, the rich metadata of traces, and the searchable content of logs within a single, efficient system.

Share this resource

Postgres managed by ClickHouse | One stack for transactions and analytics →

Email address Sign up

Follow us

Postgres managed by ClickHouse | One stack for transactions and analytics →

Comparisons

BigQuery
PostgreSQL
Redshift
Snowflake
Elastic
Splunk
OpenSearch

Partners

AWS
Azure

Stay informed on feature releases, product roadmap, support, and cloud offerings!

Email address Sign up

Star us on Github

© 2026 ClickHouse, Inc. HQ in the Bay Area, CA and Amsterdam, NL.

Trademark Privacy Security Legal Cookie policy Cookie settings

↑